FAITHDROID: Automatically Measurable Explanation Faithfulness for Cost-Aware LLM-Based Android Malware Detection under Concept Drift

Configuration Manual

MSc Practicum Internship Part 2
Cybersecurity

Vinay Appajigowda
Student ID: 24262013

School of Computing
National College of Ireland

Supervisor: Michael Pantridge

August 2026

FAITHDROID: Automatically Measurable Explanation Faithfulness for Cost-Aware LLM-Based Android Malware Detection under Concept Drift

Vinay Appajigowda
24262013

1 Purpose and Scope

This manual gives all information necessary to replicate this research. Provide a report from a clean machine, including the versions of hardware and software, the datasets acquired, and the directory. layout, configuration values, run order, expected outputs, verification commands and troubleshooting. Extended tables, full ablation output and command-level detail retained here. In the report, not in the page limited report.

2 Hardware and Operating System

Data was generated on an Apple M5 Pro (15 cores: 5 performance, 10 efficiency) with 24 GB of RAM. Running macOS 15 (Darwin 25.5.0), it has unified memory and a 16-core GPU. Any machine that can provide a service to a person. The reasoning stage is the only slow component that will be reproduced by 7B parameter model. The peak of the resident memory is less than 8 GB. The estimated disk space needed is about 430 MB: 196 MB raw KronoDroid CSVs: 7.5 MB processed Parquet splits and 17 MB cached LLM responses, 5.7 MB of data. result JSON.

3 Software Environment

Python 3.13 is required. Exact pinned versions, which the environment checker enforces:

Package	**Version**
pandas	2.3.3
numpy	2.4.6
scikit-learn	1.5.1
xgboost	3.2.0
shap	0.52.0
scipy	1.18.0
matplotlib	3.10.6
pyarrow	21.0.0

The LLM stage talks to a local Ollama server over HTTP using only the standard library, so there is no Python client dependency. Install Ollama separately and pull the three models:

```
pip install -r requirements.txt
ollama pull qwen2.5:7b     # primary reasoning model
ollama pull llama3.2:3b    # model-capacity sweep
ollama pull phi4:14b       # model-capacity sweep
python3 code/envcheck.py   # exits 1 if the stack differs from the pins
```

Reproducibility depends on the pinned stack. The splits rebuild byte-identically on any stack, but the fitted models do not; because the detector's probabilities decide which samples escalate, a different stack moves the escalated set and invalidates cached LLM responses. Results must therefore come from a single stack and must never be mixed.

4 Dataset Acquisition

KronoDroid is a public API and requires no API key. Only real-device CSVs are used (emulator) The study showed that malware file is very similar to the sample file, except that it contains 483 columns instead of 484, with a missing timestamp column. Are used in conjunction with each other; mixing the two would introduce missing values extraneously.

```
cd data/raw && git clone --depth 1 https://github.com/aleguma/kronodroid.git
cd kronodroid && for f in real_device/*.zip; do unzip -o -q "$f" -d extracted/; done
```

5 Directory Structure and Configuration

All paths resolve relative to the project root; no absolute or machine-specific path appears in the source. Configuration lives in a single module, `code/config.py`, which is the sole definition of split boundaries, target prevalence (0.10), gate parameters, model names, reliability threshold (0.70), escalation budgets and random seeds.

Path	**Contents**
`code/`	pipeline, one module per stage
`data/raw/`	KronoDroid CSVs as downloaded
`data/processed/`	five Parquet splits + metadata
`data/llm_cache/`	cached LLM responses
`results/`	all experiment output as JSON
`images/`	generated figures
`report/`	LaTeX report and build script
`config_manual/`	this manual

6 Run Order

Scripts must run in this order; `conformal.py` must precede `run_experiments.py` because the escalation ranking is read from its saved scores.

```
cd code
python3 preprocess.py          # splits            -> data/processed/
python3 baselines.py           # 4 baselines       -> results/baselines.json
python3 conformal.py           # calibration       -> results/conformal_pvalues.json
```

```
python3 routing_diagnostic.py  # signal choice     -> results/routing_diagnostic.json
python3 run_experiments.py     # all LLM configs   -> results/llm_*.json
python3 adversarial.py --split Test1
python3 adversarial.py --split Test3
python3 faithfulness.py        # EGR / CFN / AA    -> results/faithfulness.json
python3 cascade.py             # cost curve        -> results/cascade.json
python3 statistics.py          # CIs, McNemar      -> results/statistics.json
python3 figures.py             # all figures       -> images/
python3 make_tables.py         # LaTeX tables      -> report/text/generated_tables.te
python3 test_pipeline.py       # invariant tests
```

7 Caching and Checkpointing

LLM responses are cached in `data/llm_cache/`, keyed by model, prompt mode, citation-cap setting and a hash of the full prompt text. Because the prompt is part of the key, any change to prompt construction correctly invalidates the affected entries. Reruns are therefore free and an interrupted sweep resumes without repeating work. Deleting the cache forces full recomputation, which takes approximately two hours for the complete sweep of 2,962 escalated samples at six concurrent workers.

8 Verification

Three guards run over the finished artefacts and each exits non-zero on failure:

```
python3 code/envcheck.py        # installed stack vs requirements.txt
python3 code/test_pipeline.py   # invariants that would corrupt results silently
cd report && ./build.sh         # rebuilds tables from JSON, then compiles
```

The invariant tests check that no train/test hash overlap exists, that calibration strictly precedes every test window, that prevalence is pinned, that no VirusTotal-derived column is used as a feature, that every citable artifact is genuinely present in the application, that retrieval draws only from the training period, that the vectorised Jaccard matches the naive computation, that every split rebuilds identically from the raw CSVs, and that ranking ties break stably.

9 Regenerating Figures and Tables

Every figure in the report is produced by `code/figures.py` from the saved result JSON; every results table is produced by `code/make_tables.py`. The report build script invokes the table generator before compiling, so the report cannot disagree with the runs that produced it. To regenerate a single figure, run `figures.py` and copy the required PNG from `images/` into `report/images/`.

10 Known Limitations and Troubleshooting

Ollama not running. The LLM stage fails immediately if no server is listening on `localhost:11434`; start it with `ollama serve` and confirm with `ollama list`. **Cache**

misses after a code change. Expected whenever prompt construction or retrieval ordering changes; the sweep will re-query and repopulate. **Stack mismatch.** `envcheck.py` exits 1; install the pinned versions rather than mixing results across stacks. **Long runtime.** The full sweep is dominated by LLM latency (roughly 27 s per call at six-way concurrency); reduce the escalation budget for a faster smoke test. **Dataset ends in 2020**, so the protocol transfers but the specific figures are not claimed to generalise to newer samples.

)